

Software Modeling Assistance with Large Language Models

Meriem Ben Chaaben

Université de Montréal

Montreal, Canada

meriem.ben.chaaben@umontreal.ca

ABSTRACT

Software modeling requires a challenging combination of expertise in both domain knowledge and modeling formalisms. Existing methods often fail to provide effective, general modeling assistance. This research introduces a novel approach using large language models (LLMs) to enhance software modeling. Utilizing few-shot prompt learning, our method supports various modeling activities without extensive training data. Initially focusing on static and behavioral formalisms like UML diagrams, we aim to extend this to other paradigms and integrate it into the Model-Driven Engineering (MDE) pipeline. Additionally, we aim to assess productivity, model quality, and accuracy when receiving real-time, context-aware suggestions during modeling tasks.

CCS CONCEPTS

• **Computing methodologies** → *Modeling methodologies; Model development and analysis.*

KEYWORDS

Modeling formalisms, generative AI, large language models, model-driven engineering, prompt learning, user study.

ACM Reference Format:

Meriem Ben Chaaben. 2024. Software Modeling Assistance with Large Language Models. In *ACM/IEEE 27th International Conference on Model Driven Engineering Languages and Systems (MODELS Companion '24)*, September 22–27, 2024, Linz, Austria. ACM, Linz, Austria, 4 pages. <https://doi.org/10.1145/3652620.3676877>

1 RESEARCH PROBLEM AND MOTIVATION

Software modeling helps manage the complexity of engineering problems by enhancing the efficiency and effectiveness of software development through abstraction. It involves creating explicit, structured, and visual representations of real-world objects within a domain and their interconnections. However, This process is inherently complex and prone to errors due to the need for expertise in domain knowledge and modeling formalisms. Additionally, the omission of certain concepts can significantly impact the developed solutions. There is a pressing need for new tools and techniques that assist modelers in their tasks while ensuring that this assistance is efficient and effectively integrated into the modeling workflow.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for third-party components of this work must be honored. For all other uses, contact the owner/author(s).

MODELS Companion '24, September 22–27, 2024, Linz, Austria

© 2024 Copyright held by the owner/author(s).

ACM ISBN 979-8-4007-0622-6/24/09

<https://doi.org/10.1145/3652620.3676877>

Current state-of-the-art approaches often fail to provide a methodology that effectively streamlines the modeling assistance process and ensures it is general across different modeling formalisms. Our work addresses these shortcomings by introducing a systematic approach that leverages the capabilities of large language models (LLMs) to generate content that enriches a software model under construction. This novel approach relies on few-shot prompt learning, overcoming the need for extensive training of AI-based techniques, particularly given the scarcity of domain-specific datasets. It offers variable support for various modeling activities, providing valuable recommendations to software modelers.

Initially, we focus on two different formalisms: static (such as domain modeling or UML class diagrams) and behavioral (such as UML activity diagrams). Subsequently, we aim to generalize this work to include other modeling formalisms and integrate the assistance component into the Model-Driven Engineering (MDE) pipeline. Furthermore, we aim to study the usefulness of providing real-time and context-aware suggestions. We want to know if this can enhance productivity, improve the quality of the resulting models, and support modelers in creating more accurate and comprehensive software models.

2 BACKGROUND AND RELATED WORK

Assistance in Software Modeling. Model driven engineering (MDE) [22] supports modeling software systems before they are implemented and use these models for an array of crucial engineering activities, including requirements formalization [8], rapid prototyping and validation [16], formal verification of functional [10] and extra-functional properties, and code generation [14]. Domain modeling [13, 19], for instance, focuses on externalizing core domain concepts and relationships among them, typically relying on UML Class Diagrams. To alleviate the issues stemming from manual efforts, computer-aided automation has been of particular interest in software modeling [17]. Traditional software modeling assistance relies on pattern-based [15] or similarity-based [9] recommendations, and occasionally, recommendations extracted from natural language specifications [12]. Weyssow et al. [23] proposes a similarity-based, data-centered approach in which datasets of models and metamodels relevant to the target objective are transformed into textual representations to train neural networks. A similarity-based approach is due to Elkamel et al. [9], who use clustering algorithms instead of neural networks to propose UML classes. Other techniques leverage the fact that software models are eventually used for generating source code and take a reverse engineering stance. Capuano et al. [4] train a neural network with data extracted from Java projects. Their results show substantial bias in the obtained recommendations towards implementation-related concepts rather than high-level domain concepts, such as *utilities* (cf. Util classes in Java) and *controllers* (cf. Controller classes in Java).

Natural language-based techniques alleviate such biases and constitute the state-of-the-art in software modeling assistance. Arora et al. [1] propose an approach to automatically extract domain models from requirement documents by rule-based techniques. Their results show promising results with about 74 – 100% recommendations deemed eventually correct in their experiments. Regarding the usefulness of providing modeling assistance, Savary-Leblanc et al. [21] explore how these assistance features can support users during modeling tasks, and demonstrates that effective assistance must incorporate high-context information, such as domain knowledge and corporate methodology, alongside intuitive notations and tools.

Large Language Models for software modeling assistance. Large Language Models (LLMs) are AI models pre-trained on vast amounts of textual data, enabling them to capture extensive human-generated knowledge and use this encoded knowledge for generative tasks, such as creating human-like text [18]. The recent increase in the performance of LLMs has opened new avenues for their application in software engineering tasks, including software modeling. LLMs like GPT-3 have been employed in various stages of the software development lifecycle, from requirements engineering through software architecture to testing.

Recent studies have explored the potential of LLMs for providing modeling assistance. For example, regarding domain modeling, Burgu  o et al. [3] exploit language models that are trained on general natural language documents to suggest domain concepts for model completion and reach up to 62% reliability in identifying model elements. Moreover, Chen et al. [6] explore the potential of using the OpenAI model -GPT-4 for creating Goal-oriented Requirement Models (GLRs). The study shows that the powerful language model has a certain level of understanding in goal modeling averaging a score of 73.8%. However, the authors also point out that the powerful OpenAI model GPT-4 generates many generic or prompt-reflective elements that are occasionally incorrect. The use of LLMs in modeling assistance is still in its early stages, and there are challenges related to the generality of the pre-trained corpus and the need for fine-tuning. Negative results have been reported by Arulmohan et al. [2] where LLMs were outperformed by explainable natural language processing techniques. This suggests that while LLMs have potential, they may not always be the optimal solution and can fall short compared to more explainable methods. Techniques like few-shot learning, which involves providing the LLM with a small number of examples to guide its output, have shown promise in addressing these challenges. For a more structured approach, Claris   and Cabot [7] proposed using domain-specific languages (DSLs) to define platform-independent prompts, which can then be adapted to generate high-quality outputs in target AI systems.

Despite these promising approaches, there is still a need for systematic methods to integrate LLMs into domain modeling tools and evaluate their effectiveness in real-world scenarios.

3 APPROACH AND UNIQUENESS

3.1 Approach

Our approach employs few-shot prompt learning with large language models (LLMs) to assist in software modeling by providing

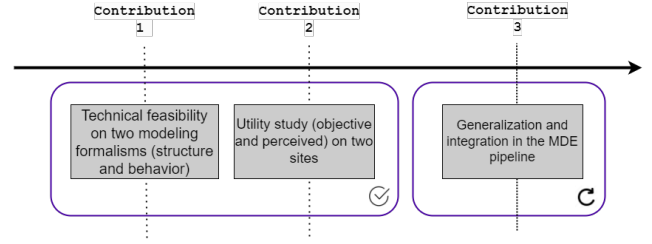


Figure 1: Overview of the three key steps of approaching the problem

real-time and context-aware suggestions for modeling elements. As illustrated in Figure 1, we present our work in three key steps. Firstly, we examine the technical feasibility of our approach on two formalisms: a structural formalism, specifically domain modeling or class diagrams, and a behavioral formalism, specifically UML activity diagrams. Secondly, we assess the real-world applicability and effectiveness of our approach through both objective and subjective evaluations. Finally, as ongoing work, we are working on generalizing this concept and integrating this assistance component into the MDE pipeline.

3.1.1 Technical faisability on two formalisms. The main goal of our approach, previously presented in our earlier paper [5], is to complete a model under construction (a.k.a. partial model) by suggesting related elements. Given a partial model, we construct a text representation that serves as input to GPT-3. The model returns a textual output following a specific pattern, which we then parse to extract relevant model elements through appropriate text transformations. Using *static diagrams* for domain modeling

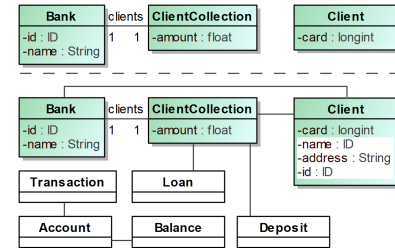


Figure 2: Example of domain model: Bank class diagram

typically involves representing domain entities, their properties, and their relationships. For instance, Figure 2 shows a UML class diagram as a partial description of a banking system. The upper part represents the partial domain model already defined by the user. Figure 3 illustrates the few-shots provided for our example. We build a query from the partial domain model by selecting 2 to 4 pairs of related classes, placing the class names in square brackets, and adding them to the prompt. The bold text in Figure 3 shows the generated output for our example.

For attribute and association suggestions, we use a similar approach, providing examples that guide the LLM in generating relevant and accurate recommendations. Each suggestion is ranked based on frequency and confidence, ensuring that the most relevant suggestions are presented to the user.

Prompt:

Generate related concepts:
 hospital: [Nurse, Staff], [Department, Room], [Nurse, patient], [Nurse, Department]
 reservationSystem: [SpecificFlight, GeneralFlight], [Airport, City], [Passenger, Plane], [Trip, Passenger], ...
 ... (Three more shots)
 Bank: [Bank, Client], [Client, Clientcollection]

Generated text:

Bank: [Bank, Client], [Client, Clientcollection], [Loan, Clientcollection], [Loan, Deposit], [Account, Balance], [Account, Transaction]

Figure 3: Prompt and generated text for class names and association prediction to complete the Class Diagram of Fig. 2

Results: We have evaluated the correctness of our approach on 30 domain models, selected manually from the dataset ModelSet [11]. The results were published in our NIER paper [5]. We achieved notable outcomes. For class name suggestions, we conducted two successive rounds of completion suggestions. In the first round (R1), the average precision was 0.57 and the recall was 0.29. After validating and integrating the suggested elements into the partial model, we conducted a second round (R2), where the recall improved to 0.45, although the precision slightly decreased to 0.56. For attribute suggestions, the average recall was 0.7 with a standard deviation of 0.4, indicating a promising result despite variability. Association name suggestions achieved an accuracy of 0.64, showing the engine’s potential in providing meaningful association names.

Since **structural diagrams** do not define sequences, any fragment can be used to generate prompts to complete a diagram. For dynamic/behavioral diagrams, such as activity diagrams [20], there are strong precedence/sequence constraints to consider (e.g., representing time). To apply prompt learning, we need to define shots and prompts that preserve these constraints. To address precedence constraints, we map the semantics of activity diagrams to a pattern that GPT-3 can understand and provide meaningful results. Our prompts are designed and parameterized to predict the next actions in a partial sequence, setting the max token hyperparameter to a relatively low number—50 in our experiments. To build the prompts, we define simple transformation rules to match the elements of the activity diagram to the appropriate keywords, forming the prompt we send to GPT-3.

3.1.2 Utility Study. Our second contribution is to empirically study various aspects of the utility of LLM-based modeling assistance. The notion of utility is closely tied to recommender systems, and can be objective or perceived. We built a tool that integrates seamlessly with a modeling environment to provide continuous support during the modeling process, more specifically *domain modeling*. The high-level overview of the tool is shown in Fig 4. We chose the *Eclipse platform*¹ to implement our tool thanks to the rapid development curve it enables through its ready-to-use modeling framework and extensible plug-in-based architecture. The *user* interacts with the models through the visual *Editor* that allows the creation of domain models using UML Class Diagrams. The *Modeling Framework* serves for representing and persisting models. The *Recommendation engine* is responsible for querying

¹<https://www.eclipse.org/>

the LLM, *GPT3* in this case, upon request or upon edit operations; collecting, organizing, and ranking recommendations; and passing this information to the editor by saving recommendations in the model. Finally, to investigate user behavior during modeling, we developed a *Logging* facility. In our tool, we implement three modes of assistance to cater to different modeling styles and preferences:

- **Automatic Mode:** Provides real-time suggestions as the modeler adds elements to the model. This mode aims to offer continuous support and keep the modeler in a steady flow of ideas.
- **On-Request Mode:** Allows the modeler to request specific suggestions at any point during the modeling process. This mode is designed for users who prefer more control over when and what assistance they receive.
- **At-End Mode:** Presents suggestions for potential enhancements after the modeler completes the initial version of the model. This mode is useful for users who prefer to first develop their ideas independently and then refine their work based on feedback.

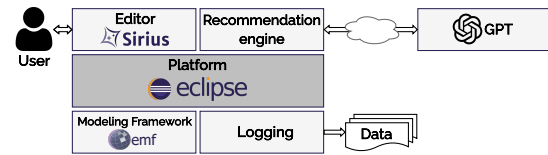


Figure 4: High-level architectural view of the tool

We use the tool to conduct a comprehensive user study with 30 subjects from two countries, Canada and Spain. Participants were asked to complete domain modeling tasks using the different modes of assistance: Automatic, On-Request, and At-End. In particular, we aim to explore the following aspects, categorized by utility:

- **Productivity**, i.e., the impact of modeling assistance on the time required to complete the domain models.
- **Contributivity**, i.e., the level of contribution of the suggestions to the resulting models.
- **Creativity**, the impact of the assistance on the diversity of the solutions produced by different modelers.

We also explored the type of assistance preferred by the modelers and their overall modeling experience.

Results: The results of the user study indicate that the mode of assistance significantly impacts the productivity, contributivity, and creativity of domain modeling. Automatic Mode enhances productivity the most, providing real-time suggestions that streamline the modeling process. However, this mode may reduce model diversity due to its continuous guidance. On-Request Mode strikes a balance by allowing modelers to seek help as needed, thus preserving creativity while still enhancing productivity. At-End Mode supports independent initial modeling and reflective refinement, maintaining diversity but showing less improvement in productivity and contributivity. User preferences suggest that different modelers have various needs for assistance, highlighting the importance of offering multiple modes to accommodate diverse working styles. The satisfaction with the tool shows the effectiveness of LLM-based assistance in enhancing domain modeling. We expect this effectiveness to extend to all formalisms as we continue to refine our approach.

3.1.3 Generalization and integration in the MDE pipeline.

Our ongoing work focuses on generalizing the modeling assistance component, extending its capabilities beyond domain models with class and activity diagrams. We aim to develop a systematic approach applicable to various modeling formalisms, ensuring the component is fully integrated into the MDE pipeline. Our goal is to automate the generation of assistants alongside editor generation. Like our initial work, we will emphasize the semantic mapping between natural language and modeling formalisms. Specifically, we plan to annotate modeling formalism constructs with semantic information such as consequence and dependency relationships, behavioral patterns, constraints and conditions, and temporal aspects. This semantic information will then be used to generate prompts, such as the one described in Figure 3. Ultimately, this integration aims to provide seamless, context-aware support throughout the software development process, making it an indispensable part of modern software engineering.

3.2 Uniqueness

Our work is unique due to its dual focus on correctness and utility. Unlike traditional studies that prioritize the accuracy of model suggestions, we emphasize the practical utility of using large language models (LLMs) to generate content and provide meaningful, actionable assistance during domain modeling. Additionally, our work is unique in its emphasis on generalizability. While our initial efforts were more ad-hoc, we are now dedicated to developing a systematic approach that relies on well-defined translation rules and patterns. This systematic approach aims to improve the generalization of our methods, enabling the LLMs to be applied effectively across various modeling formalisms. By focusing on these translation rules and patterns, we ensure that our assistance component can be integrated seamlessly into the broader MDE pipeline, providing robust support for a wide range of modeling activities.

CONCLUSION

In conclusion, this work demonstrates the potential of leveraging large language models (LLMs) to assist in software modeling through few-shot prompt learning. By focusing on static and behavioral formalisms, we have shown that LLMs can provide valuable, context-aware suggestions that significantly enhance the modeling process. Our evaluations show that different assistance modes cater to various user preferences, making our approach adaptable. Furthermore, we have laid the groundwork for generalizing this methodology to support a wider range of modeling formalisms through systematic translation rules and patterns. The integration of the modeling assistance component into the Model-Driven Engineering (MDE) pipeline ensures broad applicability and robust support for diverse modeling activities. This work highlights the promising capabilities of LLMs in enriching software modeling practices and sets the stage for future research.

REFERENCES

- [1] Chetan Arora, Mehrdad Sabetzadeh, Lionel Briand, and Frank Zimmer. 2016. Extracting domain models from natural-language requirements: approach and industrial evaluation. In *Proceedings of the ACM/IEEE 19th International Conference on Model Driven Engineering Languages and Systems*. the Association for Computing Machinery, Saint-Malo, France, 250–260.
- [2] Sathurshan Arulmohan, Marie-Jean Meurs, and Sébastien Mosser. 2023. Extracting domain models from textual requirements in the era of large language models. In *2023 ACM/IEEE International Conference on Model Driven Engineering Languages and Systems Companion (MODELS-C)*. IEEE, 580–587.
- [3] Lola Burgueño, Robert Clarisó, Sébastien Gérard, Shuai Li, and Jordi Cabot. 2021. An NLP-Based Architecture for the Autocompletion of Partial Domain Models. In *Prof. of the 33rd International Conference on Advanced Information Systems Engineering (CAiSE 2021) (Lecture Notes in Computer Science, Vol. 12751)*, Marcello La Rosa, Shazia W. Sadiq, and Ernest Teniente (Eds.). Springer, virtual event, 91–106. https://doi.org/10.1007/978-3-030-79382-1_6
- [4] Thibaut Capuano, Houari A. Sahraoui, Benoît Frénay, and Benoît Vanderose. 2022. Learning from Code Repositories to Recommend Model Classes. *Journal of Object Technology* 21, 3 (2022), 3:1–11. <https://doi.org/10.5381/jot.2022.21.3.a4>
- [5] Meriem Ben Chaaben, Lola Burgueño, and Houari Sahraoui. 2023. <http://hdl.handle.net/20.500.12004/1/C/ICSE/2023/001>.
- [6] Boqi Chen, Kua Chen, Shabnam Hassani, Yujing Yang, Daniel Amyot, Lysanne Lessard, Gunter Mussbacher, Mehrdad Sabetzadeh, and Dániel Varró. 2023. On the use of GPT-4 for creating goal models: an exploratory study. In *2023 IEEE 31st International Requirements Engineering Conference Workshops (REW)*. IEEE, 262–271.
- [7] Robert Clarisó and Jordi Cabot. 2023. Model-Driven Prompt Engineering. In *2023 ACM/IEEE 26th International Conference on Model Driven Engineering Languages and Systems (MODELS)*. IEEE, IEEE, Västerås, Sweden, 47–54.
- [8] R Geoff Dromey. 2003. From requirements to design: Formalizing the key steps. In *First International Conference on Software Engineering and Formal Methods, 2003. Proceedings*. IEEE, IEEE Computer Society, Brisbane, Australia, 2–11.
- [9] Akil Elkamel, Mariem Gzara, and Hanene Ben-Abdallah. 2016. An UML class recommender system for software design. In *2016 IEEE/ACS 13th International Conference of Computer Systems and Applications (AICCSA)*. IEEE, Agadir, Morocco, 1–8. <https://doi.org/10.1109/AICCSA.2016.7945659>
- [10] Carlos A González and Jordi Cabot. 2014. Formal verification of static software models in MDE: A systematic review. *Information and Software Technology* 56, 8 (2014), 821–838.
- [11] José Antonio Hernández López, Javier Luis Cánovas Izquierdo, and Jesús Sánchez Cuadrado. 2022. ModelSet: a dataset for machine learning in model-driven engineering. *Software and Systems Modeling* 21, 3 (2022), 967–986. <https://doi.org/10.1007/s10270-021-00929-3>
- [12] Mohd Ibrahim and Rodina Ahmad. 2010. Class Diagram Extraction from Textual Requirements Using Natural Language Processing (NLP) Techniques. In *2010 Second International Conference on Computer Research and Development*. IEEE Computer Society, NW Washington, DC United States, 200–204. <https://doi.org/10.1109/ICCRD.2010.71>
- [13] Neil Iscoe, Gerald B Williams, and Guillermo Arango. 1991. Domain modeling for software engineering. In *Proceedings-13th International Conference on Software Engineering*. IEEE Computer Society, Association for Computing Machinery, Gothenburg Sweden, 340–341.
- [14] Steven Kelly and Juha-Pekka Tolvanen. 2008. *Domain-specific modeling: enabling full code generation*. John Wiley & Sons, New Jersey, United States.
- [15] Tobias Kuschke and Patrick Mäder. 2014. Pattern-based auto-completion of UML modeling activities. In *Proceedings of the 29th ACM/IEEE international conference on Automated software engineering*. Association for Computing Machinery, Vasteras Sweden, 551–556.
- [16] Eef Moeskopf and Frits Feenstra. 2008. Introduction to rapid prototyping. In *Reverse engineering: An industrial perspective*. Springer, cardiff university, UK, 99–117.
- [17] Gunter Mussbacher, Benoit Combemale, Jörg Kienle, Silvia Abrahão, Hyacinth Ali, Nelly Bencomo, Márton Búr, Loli Burgueño, Gregor Engels, Pierre Jeanjean, Jean-Marc Jézéquel, Thomas Kühn, Sébastien Mosser, Houari Sahraoui, Eugene Syriani, Dániel Varró, and Martin Weyssow. 2020. Opportunities in Intelligent Modeling Assistance. *Softw. Syst. Model.* 19, 5 (sep 2020), 1045–1053. <https://doi.org/10.1007/s10270-020-00814-5>
- [18] Ipek Ozkaya. 2023. Application of Large Language Models to Software Engineering Tasks: Opportunities, Risks, and Implications. *IEEE Software* 40, 3 (2023), 4–8. <https://doi.org/10.1109/MS.2023.3248401>
- [19] Ruben Prieto-Díaz and Guillermo Arango. 1991. *Domain analysis and software systems modeling*. IEEE Computer Society Press, Washington, DC, United States.
- [20] James Rumbaugh, Ivar Jacobson, and Grady Booch. [n.d.]. *Unified Modeling Language Reference Manual, version 2.1.1*. <http://www.omg.org/technology/documents/formal/uml.htm>
- [21] Maxime Savary-Leblanc, Xavier Le Pallec, and Sébastien Gérard. 2023. Understanding the need for assistance in software modeling: interviews with experts. *Software and Systems Modeling* 23 (2023), 1–33.
- [22] D.C. Schmidt. 2006. Guest Editor's Introduction: Model-Driven Engineering. *Computer* 39, 2 (2006), 25–31. <https://doi.org/10.1109/MC.2006.58>
- [23] Martin Weyssow, Houari A. Sahraoui, and Eugene Syriani. 2022. Recommending metamodel concepts during modeling activities with pre-trained language models. *Software and Systems Modeling* 21, 3 (2022), 1071–1089. <https://doi.org/10.1007/s10270-022-00975-5>